



Université Internationale de Rabat
THE INNOVATIVE UNIVERSITY

HANDS-ON MPI

2026-2027

I Environnement de développement

Le langage choisi est C, utilisé conjointement aux primitives offertes par la bibliothèque de communication par échange de message MPI. L'acronyme MPI signifie Message Passing Interface, spécification d'environnements de programmation pour systèmes à mémoire distribuée. Dans le cadre des TPs, nous utiliserons l'implémentation MPI.

Exécuter un programme MPI nécessite l'inclusion des fichiers d'entête dans lesquels se trouvent les définitions des différentes primitives de MPI. Il existe 6 commandes de base :

- *MPI_Init* qui initialise un bloc de calcul utilisant MPI.
- *MPI_Finalize* qui termine un bloc de calcul MPI.
- *MPI_COMM_size* qui détermine le nombre de processus qui seront engagé dans un calcul.
- *MPI_COMM_rank* qui détermine l'identificateur d'un processus.
- *MPI_Send* primitive d'envoi de messages.
- *MPI_Recv* primitive de reception de messages.

Fonctions de base MPI

L'appel à des fonctions MPI doit obligatoirement être précédé d'un appel à *MPI_Init* et se terminer par un appel à *MPI_Finalize*. Un programme MPI se décompose donc généralement de la manière suivante :

```
#include <mpi.h>
int main(int argc, char *argv[]){
    MPI_Init(&argc, &argv);
    /* CALCUL */
    MPI_Finalize();
}
```

Pour mesurer les performances d'un programme, il est utile de chronométrer son temps d'exécution. On peut faire appel pour cela à la fonction *MPI_Wtime()* qui retourne le temps écoulé en secondes dans le processus courant.

Les communicateurs

Tous les appels à des fonctions MPI se basent sur la notion de communicateurs. Un communicateur est un paramètre utilisé pour spécifier à la fonction appelée l'ensemble des processus auxquels s'adresse l'appel. Un communicateur particulier appelé *MPI_COMM_WORLD* inclut tous les processus MPI en cours d'exécution liés au processus appelant (see Figure 1).

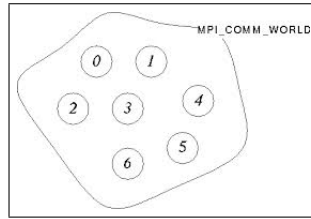


FIGURE 1 – Communicateurs

Les primitives *MPI_COMM size()* et *MPI_COMM rank()* permettent à chaque processus de connaître son environnement d'exécution, il obtient son propre identifiant et le nombre de processus qui exécutent le même programme que lui. Par exemple :

```
int size;
MPI_Comm_size(MPI_COMM_WORLD, &size);
```

aura pour effet de stocker le nombre de processus liés au communicateur *MPI_COMM_WORLD* (c'est à dire tous les processus) et de stocker ce nombre dans la variable *size*.

À chaque processus est associé un identifiant appelé *rang* ; ce *rang*, variant de 0 à (*size-1*), est propre à chaque communicateur. Un processus peut connaître son *rang* au sein d'un communicateur en faisant appel à la fonction *MPI_Comm_rank* :

```
int MPI_Comm_rank(MPI_Comm comm, int *rank)
```

Le *rang* permet au programmeur d'identifier la source et la destination d'un message ou encore de séparer l'ensemble du calcul entre les différents processus.

Communications point à point entre processus

Les processus ont aussi besoin de communiquer entre eux. Par exemple, supposons qu'un processus *p1* veuille communiquer un résultat à un processus *p2*, il va utiliser un mode de communication de type point-à-point, c'est-à-dire un envoi de message simple. Pour qu'un processus émetteur puisse envoyer un message à un processus destinataire, différentes informations sont nécessaires (see Figure 2) :

- le rang du processus émetteur dans le communicateur utilisé ;
- le rang du processus destinataire dans le communicateur utilisé ;

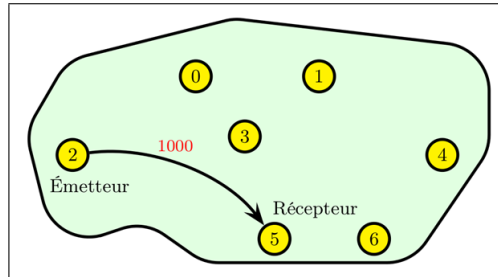


FIGURE 2 – Communications point à point

- l'étiquette du message (valeur entière choisie par le programmeur);
- le nom du communicateur.

Les primitives utilisées pour ce type de communication sont :

`MPI_Send(void *,int, MPI Datatype, int, int, MPI Comm) :`

1. adresse du buffer d'émission,
2. nombre d'éléments de type datatype à envoyer/recevoir
3. type des éléments à envoyer/recevoir (parmi `MPI_CHAR`, `MPI_SHORT`, `MPI_INT`, `MPI_LONG`, `MPI_FLOAT`, `MPI_DOUBLE`, `MPI_LONG_DOUBLE`);
4. destinataire (il s'agit de son rang),
5. tag (permet de différencier les messages), éventuellement `MPI_ANY_TAG`
6. communicateur (e.g. `MPI_COMM_WORLD`);

`MPI_Recv(void *, int, MPI Datatype, int, int, MPI Comm, MPI Status*) :`

1. adresse du buffer de reception,
2. nombre d'éléments,
3. type des éléments,
4. source, rang du processus dont provient le message,
5. tag, permet de différencier les messages à la reception,
6. status : état de la communication. Peut être ignoré avec `MPI_STATUS_IGNORE`

La fonction `MPI_Get_processor_name` permet d'identifier le nom de la machine exécutant le processus courant.

```
int namelen;
char processor_name[MPI_MAX_PROCESSOR_NAME];
MPI_Get_processor_name(processor_name,&namelen);
```

II Installation de MPI

Exécutez les commandes suivantes pour installer MPI : *sudo apt-get update*, puis *sudo apt-get install mpich*. Cette installation nécessite que le compilateur *gcc* soit déjà installé.

Pour compiler les programmes utilisant la bibliothèque MPI, il suffit d'utiliser le compilateur fourni par MPI : *mpicc*. Les paramètres sont similaires à ceux de *gcc*.

```
mpicc hello_c.c -o hello_c
```

L'exécution de programme MPI est réalisée à l'aide du programme *mpirun*.

```
mpirun -np 3 hello_c
```

- np <#> : indique le nombre de processus à utiliser

L'exemple suivant présente un simple programme de Hello World distribué utilisant MPI. Ce code permet de demander à chaque processus d'afficher le texte "Hello World" ainsi que son rang et le nombre total de processus.

```
#include <stdio.h>
#include <mpi.h>
int main(int argc, char* argv[])
{
    int rank, size;
    // Initialisation de la biblioth'que MPI
    MPI_Init(&argc, &argv);
    // Quel est mon rang ?
    MPI_Comm_rank(MPI_COMM_WORLD, &rank);
    // Combien de processus dans l'application ?
    MPI_Comm_size(MPI_COMM_WORLD, &size);
    printf("Hello, world, I am %d of %d\n", rank, size);
    // Finalisation du programme
    MPI_Finalize();
    return 0;
}
```

III Lab1

L'objectif de ce TP est de se familiariser avec la programmation parallèle en utilisant MPI et appliquer ces notions à des opérations sur vecteurs et matrices (TP2).

Exercice 1

Modifiez le programme précédent pour que les processus pairs n'affichent pas le même message que les processus impairs.

- Soit par exemple pour les processus de rang pair un message du genre :
Je suis le processus pair de *rang* M
- Et pour les processus de rang impair un message du genre : Je suis le processus impair de *rang* N

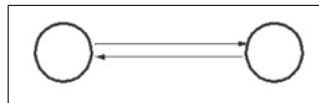


FIGURE 3 – Communications entre deux processus

Exercice 2

Ecrivez un programme dans lequel le processus 0 envoie un tableau de 10 réels au processus 1 (Figure 4). Le processus 0 remplira le tableau, et le processus 1 affichera ses valeurs. Dans ce cas, on fera uniquement un ping (envoi d'un message du processus 0 au processus 1). Par exemple, envoyer un message contenant 1000 nombres réels du processus 0 vers le processus 1 (il s'agit alors seulement d'un ping).

Modifiez le programme précédent pour que 1 renvoie les données modifiées à 0 (Figure 5). Modifiez ensuite le programme pour prendre le temps de communication (grossissez la taille du message). Dans ce cas, on enchaînera le pong après le ping (le processus 1 renvoyant le message reçu du processus 0). Le processus 1 renvoie le message reçu au processus 0 et mesure le temps de communication à l'aide de la fonction `MPI_WTIME()`.

Modifiez le programme précédent pour générer une courbe de temps de communications (Figure 6). Dans ce cas, on effectuera une répétition du ping-pong en faisant varier à chaque fois la taille du message à échanger, c'est-à-dire on fait varier la taille du message dans une boucle et mesure les temps de communication respectifs.

Exercice 3

Ecrire un programme permettant aux processus organisés en structure d'anneau (ring) de faire circuler entre eux un message. En fait, le processus prin-

```
#include "mpi.h"
#include <stdlib.h>
#include <stdio.h>

int main(int argc, char *argv[]) {
    int rang, iter;
    int nb_valeurs=1000;
    int etiquette=99;
    double valeurs[nb_valeurs];
    MPI_Status statut;

    /* ? */

    if (rang == 0) {
        for (iter = 0; iter<nb_valeurs; iter++)
            valeurs[iter] = rand() / (RAND_MAX + 1.);
        /* ? */
    } else if (rang == 1) {
        /* ? */
        printf("Moi, processus 1, j'ai recu %d valeurs (derniere = %g)"
               "du processus 0.\n", nb_valeurs, valeurs[nb_valeurs-1]);
    }

    /* ? */
    return 0;
}
```

FIGURE 4 – Exercice 3 : Programme 1

```
#include "mpi.h"
#include <stdlib.h>
#include <stdio.h>

int main(int argc, char *argv[]) {
    int rang, iter;
    int nb_valeurs=1000;
    int etiquette=99;
    double valeurs[nb_valeurs];
    MPI_Status statut;
    double temps_debut, temps_fin;

    /* ? */

    if (rang == 0) {
        for (iter = 0; iter<nb_valeurs; iter++)
            valeurs[iter] = rand() / (RAND_MAX + 1.);
        temps_debut=MPI_Wtime();
        /* ? */
        temps_fin=MPI_Wtime();
        printf("Moi, processus 0, j'ai envoye et recu %d valeurs"
               "(derniere = %g) du processus 1 en %f secondes.\n",
               nb_valeurs, valeurs[nb_valeurs-1], temps_fin-temps_debut);
        /* ? */
    }

    /* ? */
    return 0;
}
```

FIGURE 5 – Exercice 3 : Programme 2

```

#include "mpi.h"
#include <stdlib.h>
#include <stdio.h>

int main(int argc, char *argv[]) {
    int rang, iter, iter2;
    int nb_tests=0;
    int nb_valeurs[nb_tests];
    int nb_valeurs_max=7000000;
    int etiquette=0;
    double *valeurs;
    MPI_Status statut;
    double temps_debut, temps_fin;

    /* ? */

    nb_valeurs[0]=0, nb_valeurs[1]=1, nb_valeurs[2]=10, nb_valeurs[3]=100;
    nb_valeurs[4]=1000, nb_valeurs[5]=10000, nb_valeurs[6]=100000;
    nb_valeurs[7]=1000000, nb_valeurs[8]=7000000;

    /* ? */

    valeurs = (double *) malloc(nb_valeurs_max*sizeof(double));

    for(iter=0; iter<nb_tests; iter++) {
        if (rang == 0) {
            for (iter2 = 0; iter2<nb_valeurs[iter]; iter2++)
                valeurs[iter2] = rand() / (RAND_MAX + 1.);
            temps_debut=MPI_Wtime();
            /* ? */
            temps_fin=MPI_Wtime();
            if (nb_valeurs[iter] != 0) {
                printf("Hol, processus 0, j'ai envoye et recu %8d valeurs"
                    "(derniere = %4.2f) du processus 1 en %8.6f secondes, soit "
                    "avec un debit de %7.2f Mo/s.\n",
                    nb_valeurs[iter], valeurs[nb_valeurs[iter]-1],
                    temps_fin-temps_debut,
                    2.*nb_valeurs[iter]*8/1000000./((temps_fin-temps_debut)));
            } else
                printf("Hol, processus 0, j'ai envoye et recu %8d valeurs en %8.6f "
                    "secondes, soit avec un debit de %7.2f Mo/s.\n",
                    nb_valeurs[iter], temps_fin-temps_debut,
                    2.*nb_valeurs[iter]*8/1000000./((temps_fin-temps_debut)));
        } else if(rang == 1) {
            /* ? */
        }
    }

    /* ? */
    return 0;
}

```

FIGURE 6 – Exercice 3 : Programme 3

principal (possédant le rang 0) initialise la variable message à 10 puis il fait passer le message au processus suivant. Ensuite, chaque processus fait passer le message au processus d'après de manière à ce que le message fasse le tour de l'anneau. À chaque passage du message, au niveau du processus principal, celui-ci décrémente la valeur du message. Lorsque le message atteint la valeur 0, les processus transmettent une dernière fois le message avant de s'arrêter. Modifier ce programme pour faire circuler le rang de chaque processus et afficher la somme des rangs de tous les processus. Dans la Figure 7, le message est envoyé de 0 à 1, puis de 1 à 2, puis de 2 à 3. La somme des rangs est 6.

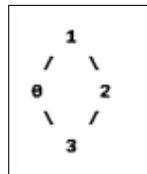


FIGURE 7 – Exemple d'anneau

Exercice 4

Ecrire un programme permettant à un noeud maître d'envoyer un entier lu sur l'entrée standard à chaque esclave, comme le montre la Figure .

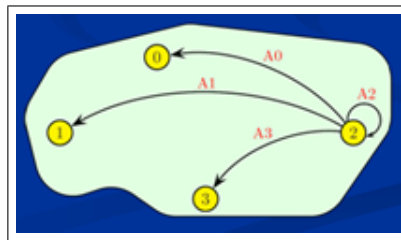


FIGURE 8 – Exemple de communication

IV Lab2

L'objectif de ce TP est d'écrire des programmes, en utilisant MPI, permettant de réaliser les opérations suivantes :

- Addition de deux vecteurs (Exo1)
- Multiplication matrice x vecteur (Exo2)

- Multiplication matrice x matrice (Exo3)

Il s'agit en particulier :

- Implémenter chaque opération en utilisant MPI pour paralléliser les calculs.
- Mesurer les temps d'exécution en fonction de la taille des vecteurs/-matrices et du nombre de processus.
- Comparer les performances et analyser l'influence de la distribution des données et du parallélisme.

Exercice 1

Ecrivez un programme MPI permettant d'additionner deux vecteurs. Vous devriez voir le schéma de décomposition des données pour un processus exécutant comme suit :

- Initialisez d'abord le nombre de processus (désignés lorsque vous exécutez le code).
- Déterminez le rang de chaque processus.
- Initialiser les tableaux de données.
- Répartir les tableaux et le calcul.
- Combiner les résultats.
- Terminer tous les processus.

Exercice 2

Ecrivez un programme permettant de multiplier un vecteur par une matrice.

Exercice 3

Ecrivez un programme permettant de multiplier deux matrices.

V TP à rendre via la plateforme

Documents à fournir (via la plateforme, voir dates de rendus) : rapport (max 5 pages), les sources (fichier .c) et les jeux de tests. Ils seront rendus dans une archive (nom-prenom-TPx.zip/gz).

VI Projets

L'objectif est de développer une version parallèle en MPI des deux algorithmes suivants (voir les fiches de projets pour plus de détails :)

- Projet 1 : Longest Common Subsequence algorithm.
- Projet 2 : RSA, a public-key cryptographic algorithm.

Date de rendu des projets : *mentionnée dans la plateforme*

Organisation

Travail individuel.

Critères d'évaluation

Les critères qui seront utilisés pour évaluer le travail rendu sont les suivants :

- La qualité du code, càd le code rendu devra être lisible et commenté.
- La qualité et la pertinence des tests.
- La qualité du rapport rendu.

Documents à fournir par email :

Rapport (max 10 pages), les sources et les jeux de tests seront rendus, via la plateforme, dans une archive (nom-prenom-Projet.zip/gz).